Variational Autoencoders

Métodos Generativos, curso 2026-2027

Guillermo Iglesias, guillermo.iglesias@upm.es
Edgar Talavera Muñoz, e.talavera@upm.es

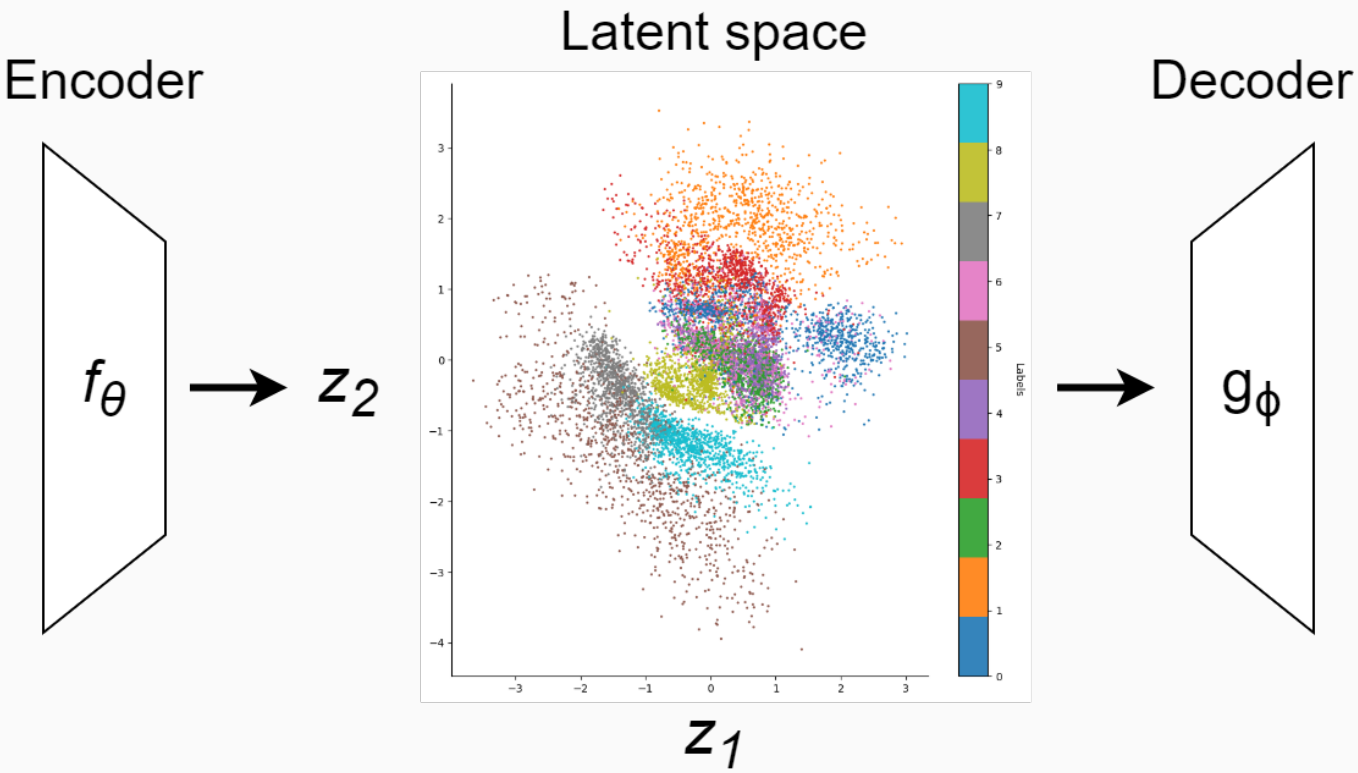
7 de noviembre de 2026

Escuela Técnica Superior de Ingeniería de Sistemas Informáticos | UPM



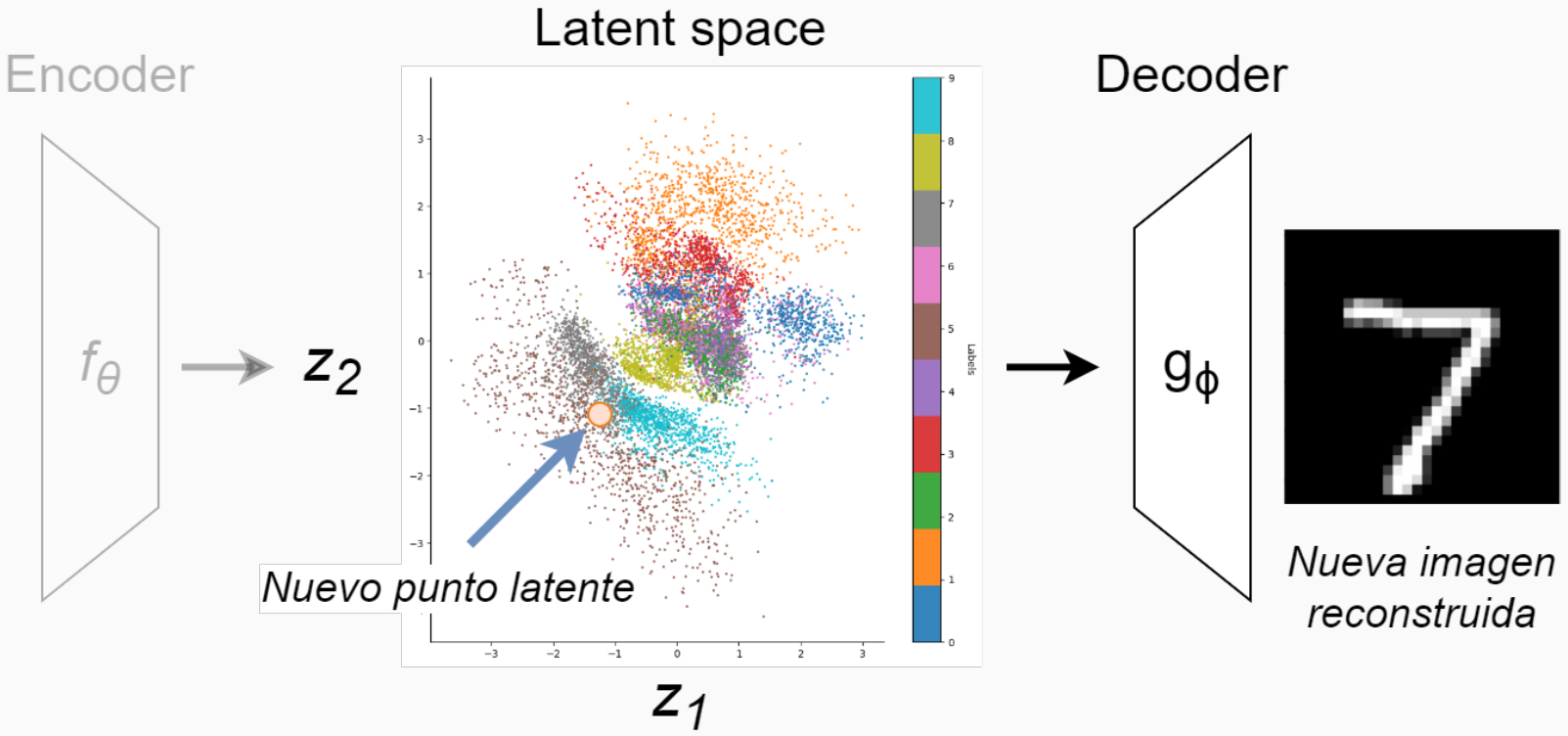
Generación en AE clásicos

Un Autoencoder clásico aprende a mapear una entrada *x* a un único punto *z* en el espacio latente.



Generación en AE clásicos

Si queremos generar un dato nuevo, la idea intuitiva sería inventarnos un vector z_{nuevo} al azar y pasarlo por el Decodificador.



Espacio latente en AE

Sin embargo, esto falla estrepitosamente:

- **Espacio discontinuo:** La red solo sabe decodificar los puntos exactos que vio durante el entrenamiento. Los espacios intermedios están ”vacíos” o generan ruido.
- **Distribución desconocida:** No sabemos qué forma tiene la distribución de probabilidad $p(z)$. Si elegimos un z aleatorio, es muy probable que caigamos fuera de la *variedad* (manifold) de los datos reales.

Motivación

Pensemos en un ejemplo sencillo: la reconstrucción de imágenes del dataset MNIST.

¿Cómo pensáis que será el espacio latente (representación en el *bottleneck*)?

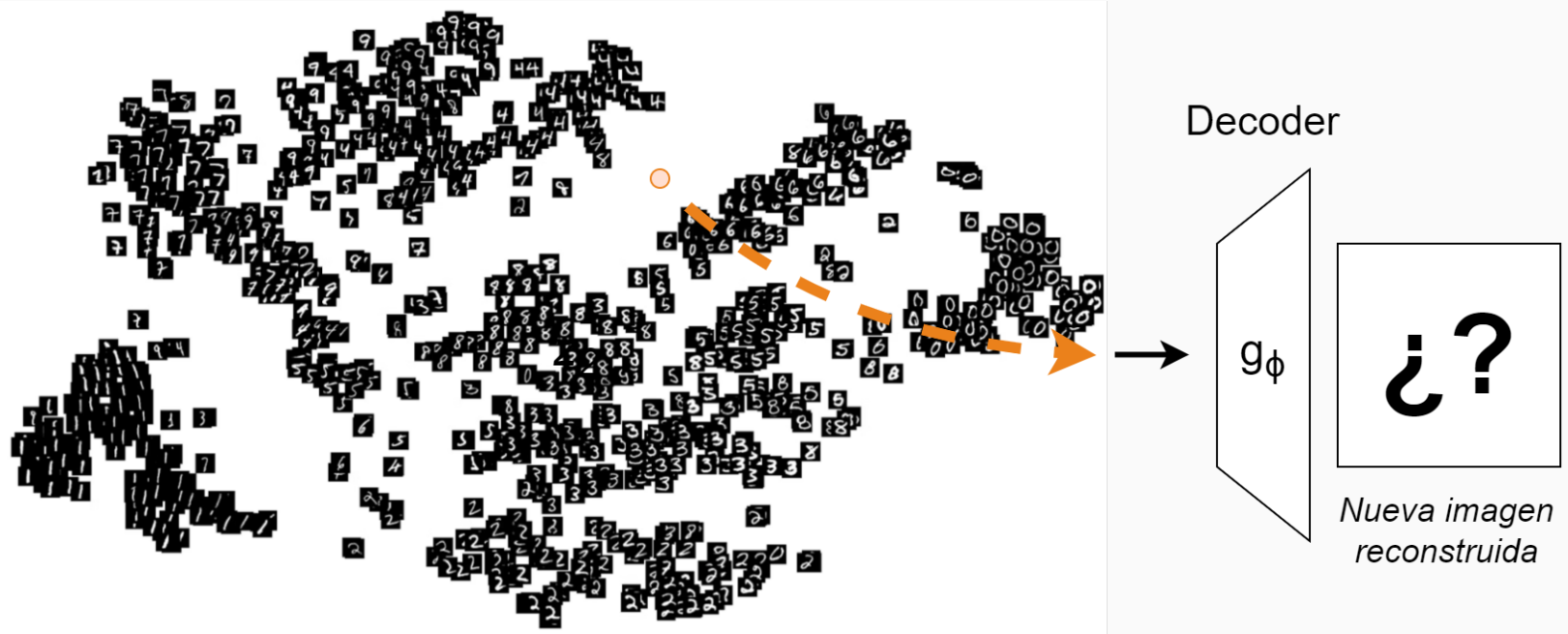


[1]

Motivación

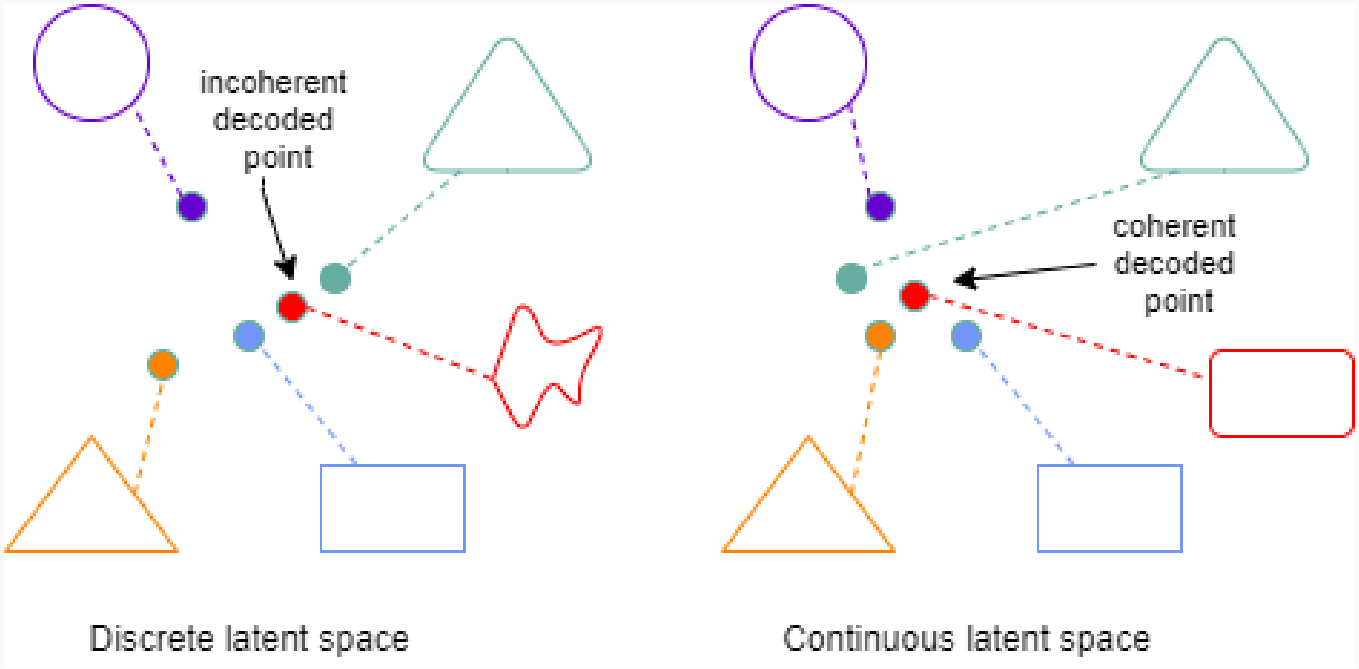
Al no tener un espacio latente continuo, tendremos problemas cuando la entrada sea ligeramente distinta a los datos con los que se entrenó el autoencoder:

¿Qué ocurrirá cuando la entrada sean imágenes que generen códigos latentes entre medio de las muestras de entrenamiento?



Motivación

Esta imagen lo muestra de forma intuitiva:

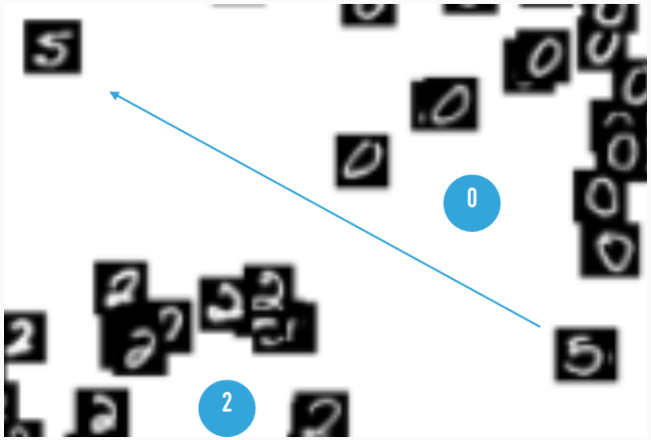


Ejemplo de problemas al generar *muestrear* de un espacio latente no continuo (fuente).

Motivación

Se busca un espacio latente continuo y ordenado:

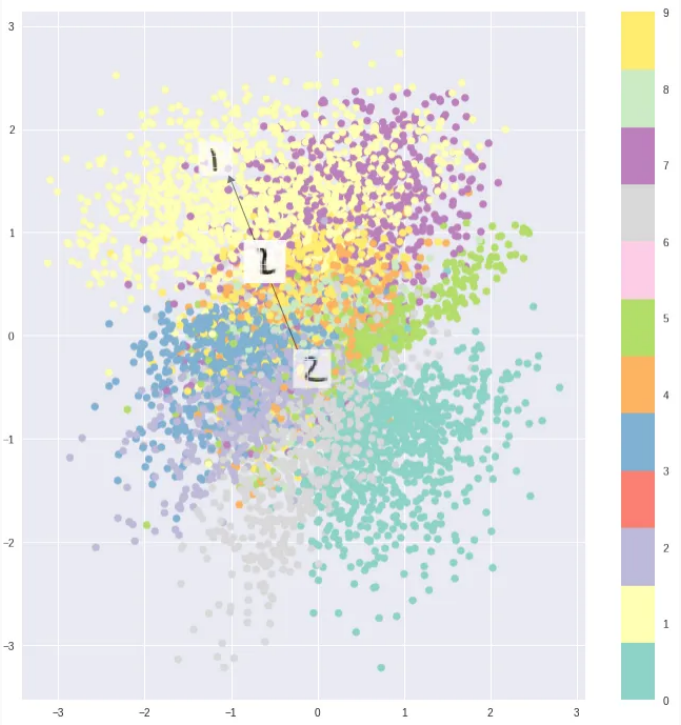
- En el que poder obtener muestras parecidas a los datos de la entrada aunque no coincidan exactamente con alguno de los de entrenamiento.
- En el que poder interpolar entre distintos espacios latentes para obtener nuevas muestras.



Ejemplo de problemas al generar nuevas muestras (fuente).

Motivación

Ejemplo de interpolación con un espacio latente **contínuo** y **ordenado**:

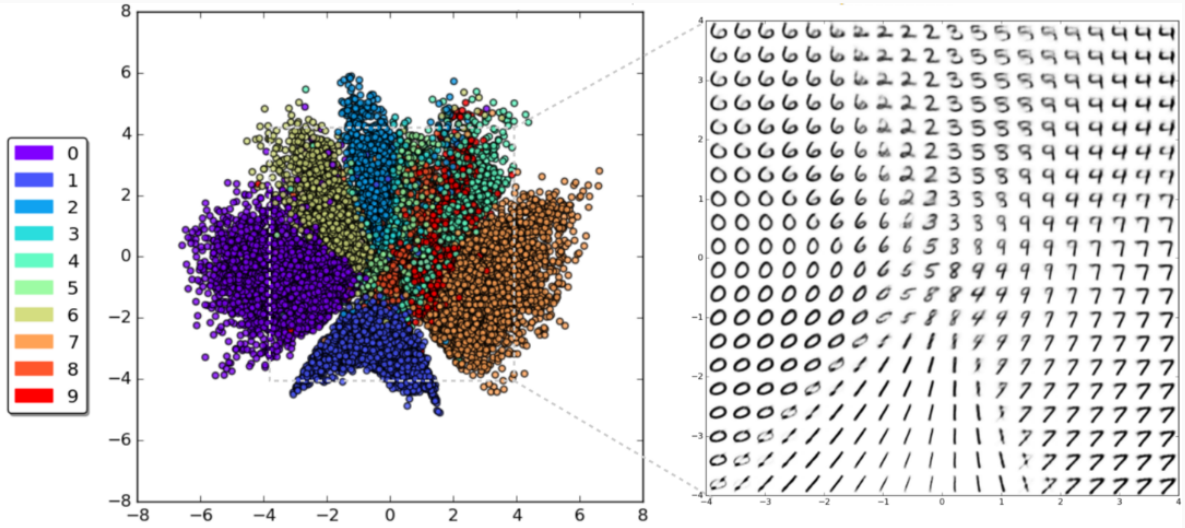


Ejemplo de interpolación (fuente).

Motivación

¿Cómo lo logramos?

Modificando ligeramente la arquitectura del AE para conseguir un espacio latente **continuo** y **ordenado**.



Ejemplo de espacio latente **continuo** y **ordenado** de las muestras generadas a partir de su muestreo. (fuente).

Variational Autoencoders (VAEs)

¿Qué son los Auto-Encoders Variacionales (VAEs)? Son una variante de los autoencoders [2] que permiten la generación de datos sintéticos.

- Combinan redes neuronales con distribuciones de probabilidad.
- Permiten que los datos generados sigan el mismo patrón que los datos de entrada.

Así, la red aprende los parámetros de una distribución de probabilidad.

- Construyen explícitamente un **espacio latente continuo** y **ordenado**.
- No una función arbitraria como en las redes neuronales convencionales.

VAEs

Propuestos por Kingma y Welling (2013) [2], los Variational Autoencoders (VAE) resuelven este problema con un cambio de paradigma fundamental:

En lugar de mapear una entrada x a un vector determinista z, el Codificador mapea la entrada a una distribución de probabilidad.

- El Codificador $f_\theta(z|x)$ no devuelve un punto, sino los parámetros de una distribución Gaussiana: un vector de medias μ y un vector de varianzas σ^2.
- El vector latente z que pasará al Decodificador se muestrea aleatoriamente de esa distribución: $z \sim \mathcal{N}(\mu, \sigma^2)$.

Esta aleatoriedad fuerza a que el espacio latente sea continuo (suave), ya que puntos cercanos deben decodificarse en imágenes similares.

El ”Prior” y la Generación

Para poder generar datos desde cero sin depender de una imagen de entrada, necesitamos establecer un Prior (una distribución base) para todo el espacio latente.

En los VAEs, asumimos que la distribución global de todo el espacio latente sigue una Gaussiana Estándar:

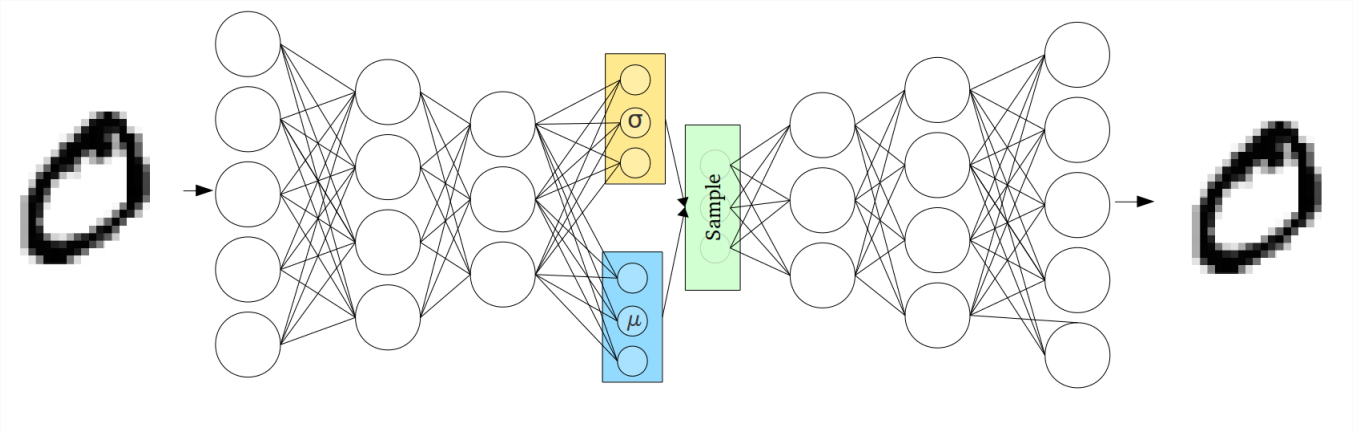
$$p(z) = \mathcal{N}(0, I)$$

¿Por qué esto es brillante para la generación? Porque si logramos que nuestro espacio latente se organice con esta forma matemática, generar una nueva imagen es tan sencillo como:

1. Generar un vector z usando un generador de números pseudoaleatorios estándar $\mathcal{N}(0, I)$.
2. Pasar ese vector por el Decodificador $q_\phi(x|z)$.

Estructura

El espacio latente está definido por **dos vectores de tamaño** *n*:

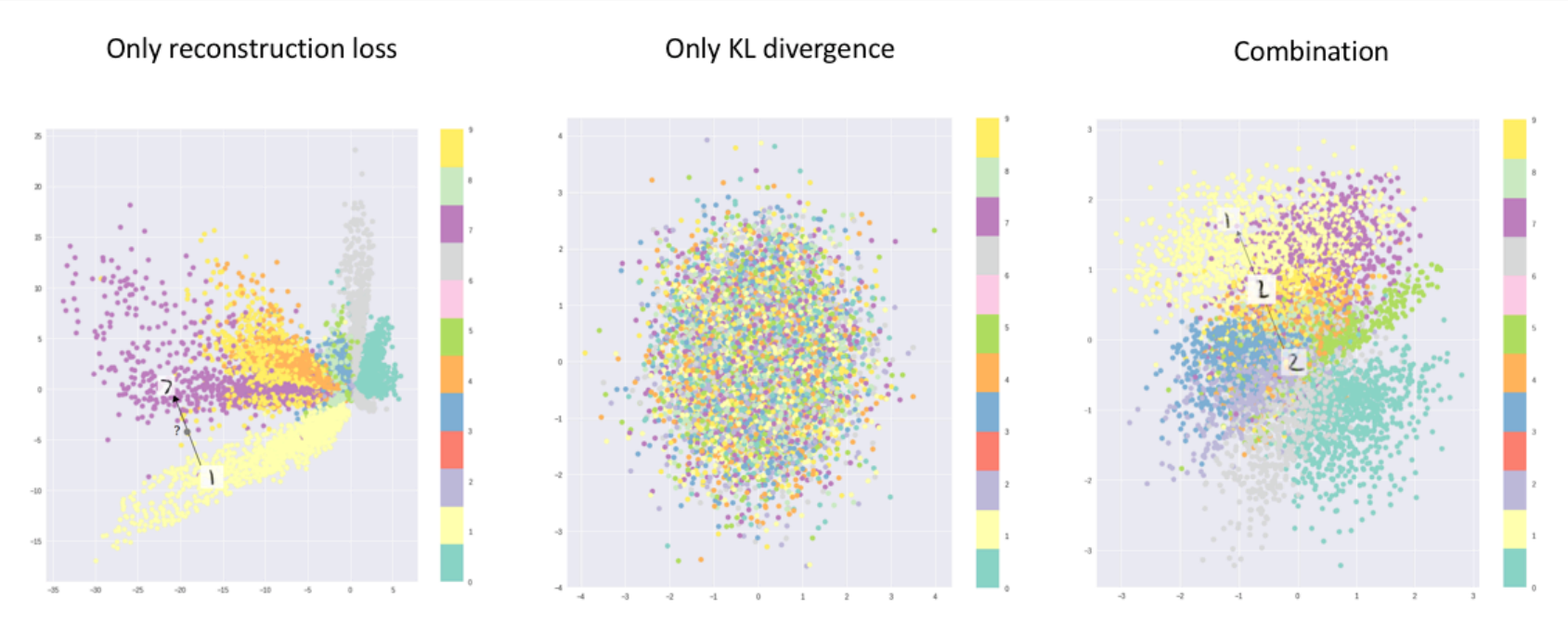


Luego debemos ajustar las funciones de pérdida individualmente de tal manera que:

- Una **función de pérdida tradicional** que calcula la **diferencia** con el objeto generado.
- La **divergencia KL (Kullback-Leibler)** entre la distribución latente aprendida y la distribución anterior (*prior distribution*), que actúa como término de regularización.
- Se suele usar $\mathcal{N}(0, 1)$

Estructura

¿Por qué necesitamos las pérdidas de reconstrucción y la divergencia KL?



Ejemplo con diferentes términos de la función de pérdidas (fuente).

La Función de Pérdida

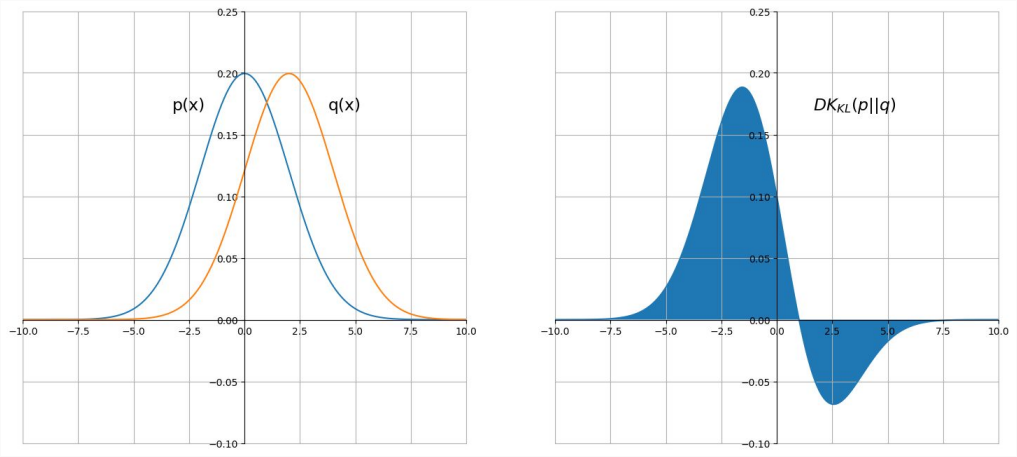
Para lograr que la red reconstruya las imágenes y, al mismo tiempo, organice su espacio latente como una Gaussiana estándar, la función de pérdida se divide en dos términos:

$$\mathcal{L}(\theta, \phi) = \underbrace{-\log g_\phi(f_\theta(x))}_{\text{1. Error de Reconstrucción}} + \underbrace{(f_\theta(z|x) \parallel q_\phi(z))}_{\text{2. Divergencia KL}} \tag{1}$$

- **Error de Reconstrucción:** Asegura que el Decodificador sepa recuperar la imagen original (como en un AE clásico, suele ser **MSE** o **BCE**).
- **Divergencia Kullback-Leibler (KL):** Mide la diferencia entre la distribución predicha por el codificador $\mathcal{N}(\mu, \sigma^2)$ y la distribución deseada $\mathcal{N}(0, I)$.

Divergencia KL (I)

Mide la diferencia entre dos distribuciones de probabilidad.



Por ejemplo, en las distribuciones de la figura tenemos dos distribuciones:

- Una distribución normal y conocida $p(x)$.
- Una distribución normal y desconocida $q(x)$.

Es una **divergencia, no una distancia**, ya que no es simétrica.

Divergencia KL (II)

Forzando una distribución normal estándar ($\mu = 0$ y $\sigma = 1$) para nuestra distribución de datos, tenemos que la divergencia KL se puede calcular como:

$$KL = \sum_{i=1}^{n} \sigma_i^2 + \mu_i^2 - log(\sigma_i) - 1$$

Nuestra función de pérdida consistirá en dos términos:

- Función de pérdida tradicional $\mathcal{L}_r$: Ajustará los datos de salida.
- Función de divergencia *KL*: Ajustará el espacio latente a la distribución estándar.

Por lo tanto, la expresión de la función de pérdida será:

$$\mathcal{L} = \mathcal{L}_r(x, \hat{x}) + \mathcal{L}_{\text{KL}}(z, \mathcal{N}(0, I)) \tag{2}$$

Entendiendo la Divergencia KL en VAEs

El término de la Divergencia KL actúa como un **regularizador**.

- Sin la divergencia KL, la red haría que las varianzas σ^2 tendieran a 0 para eliminar la aleatoriedad, volviendo a ser un Autoencoder clásico y arruinando el espacio continuo.
- La divergencia KL penaliza a la red si la media μ se aleja de 0 o si la varianza σ^2 se aleja de 1.
- Esto fuerza a que las distribuciones de todas las imágenes de entrenamiento se superpongan y se empaqueten de forma densa alrededor del origen $(0, 0)$, creando un espacio sin "agujeros" ideal para generar.

El Reparameterization Trick

Existe un problema grave de ingeniería: el proceso de muestrear $z \sim \mathcal{N}(\mu, \sigma^2)$ es una operación estocástica (aleatoria). **No se puede calcular el gradiente** a través de un nodo aleatorio, lo que rompe la *retropropagación* (backpropagation).

Kingma y Welling introdujeron el Truco de Reparametrización: Extraemos la aleatoriedad a un nodo externo ϵ que no requiere gradientes.

En lugar de muestrear z directamente, lo calculamos como:

$$z = \mu + \sigma \odot \epsilon \tag{3}$$

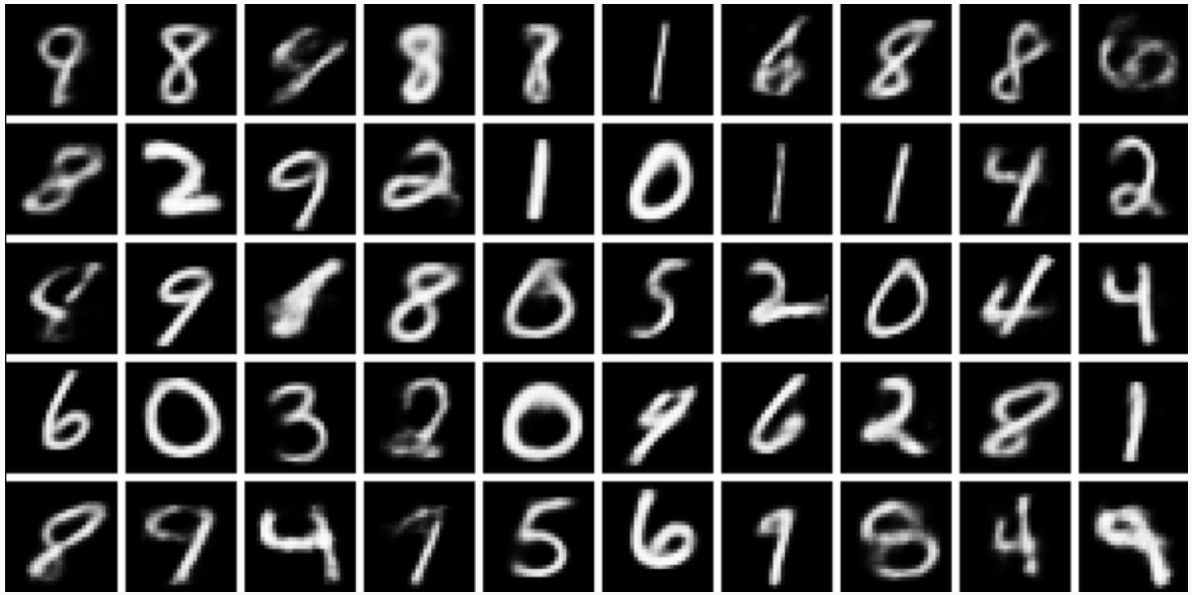
Donde $\epsilon \sim \mathcal{N}(0, I)$ es un ruido constante durante ese paso[1].

[1] Ahora las derivadas parciales respecto a μ y σ fluyen perfectamente

El problema del control en la generación

El proceso de generación de un VAE estándar es aleatorio.

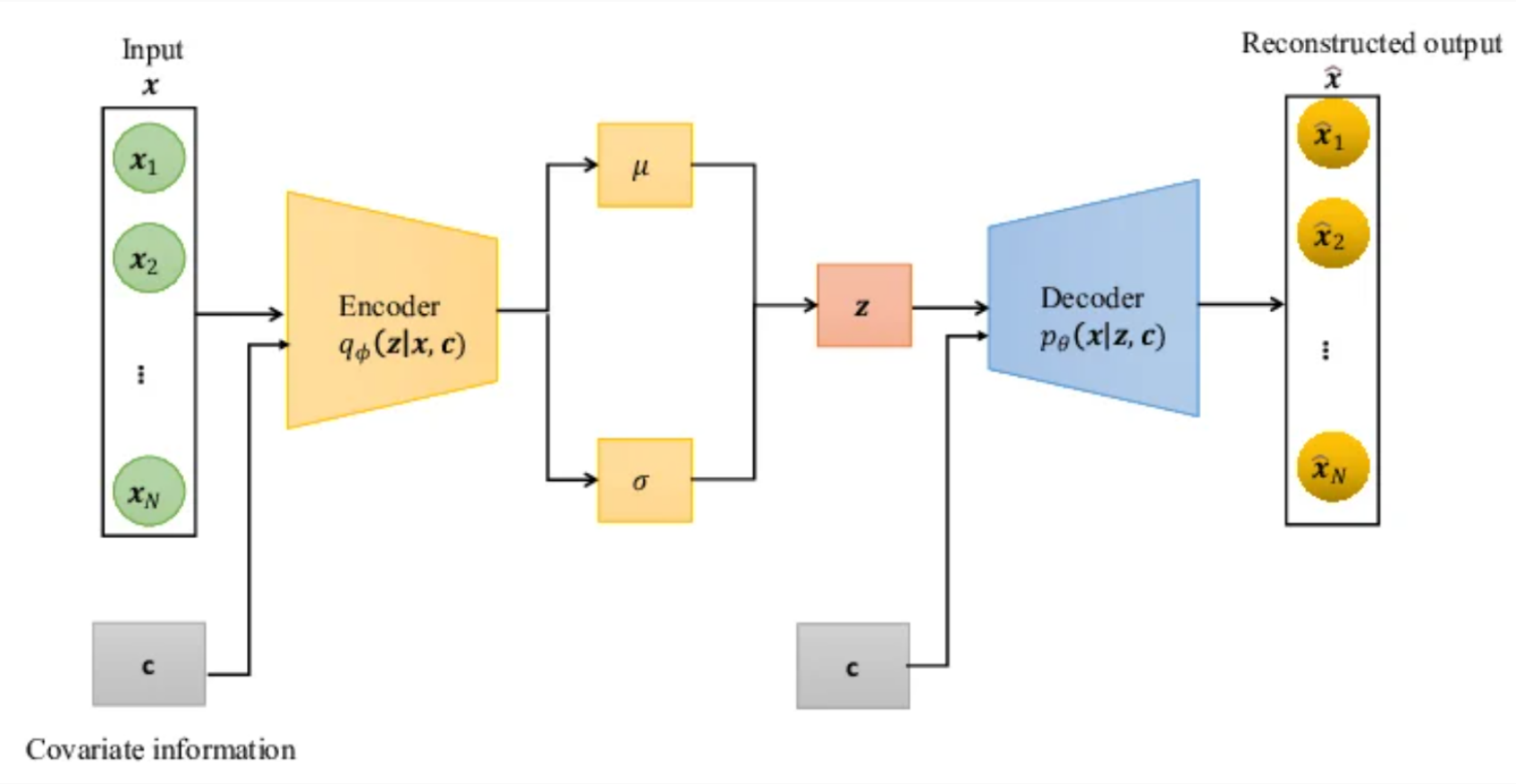
- Si entrenamos un VAE con el dataset MNIST (dígitos del 0 al 9), no tenemos forma de pedirle a la red que genere específicamente un ”3” o un ”7”.
- La red generará un número aleatorio dependiendo de dónde caiga el vector z.



[3]

cVAE

La solución de los Conditional VAEs (CVAEs) [4] es inyectar una variable condicional c (como una etiqueta de clase) en el modelo.



[5]

cVAE

Esto transforma las distribuciones de probabilidad subyacentes:

- Codificador Condicionado: Ahora estima la media y varianza observando tanto la imagen x como su clase c.

$$q_\phi(z|x,c) = \mathcal{N}(\mu_\phi(x,c), \sigma^2_\phi(x,c))$$

- Decodificador Condicionado: Reconstruye la imagen observando tanto el vector latente z como la clase c.

$$p_\theta(x|z,c)$$

El Prior (la distribución base) también se condiciona teóricamente $p(z|c)$, aunque en las implementaciones más comunes se asume que el espacio latente global sigue siendo independiente de la clase: $p(z) = \mathcal{N}(0, I)$.

cVAE: Función de Pérdida

La función de pérdida se adapta para reflejar la condición c en cada paso del proceso:

$$\mathcal{L}_{cVAE} = \underbrace{-\log p_\theta(x|z,c)}_{\text{Error de Reconstrucción condicionado}} + \underbrace{(q_\phi(z|x,c) \parallel p(z|c))}_{\text{Divergencia KL}} \tag{4}$$

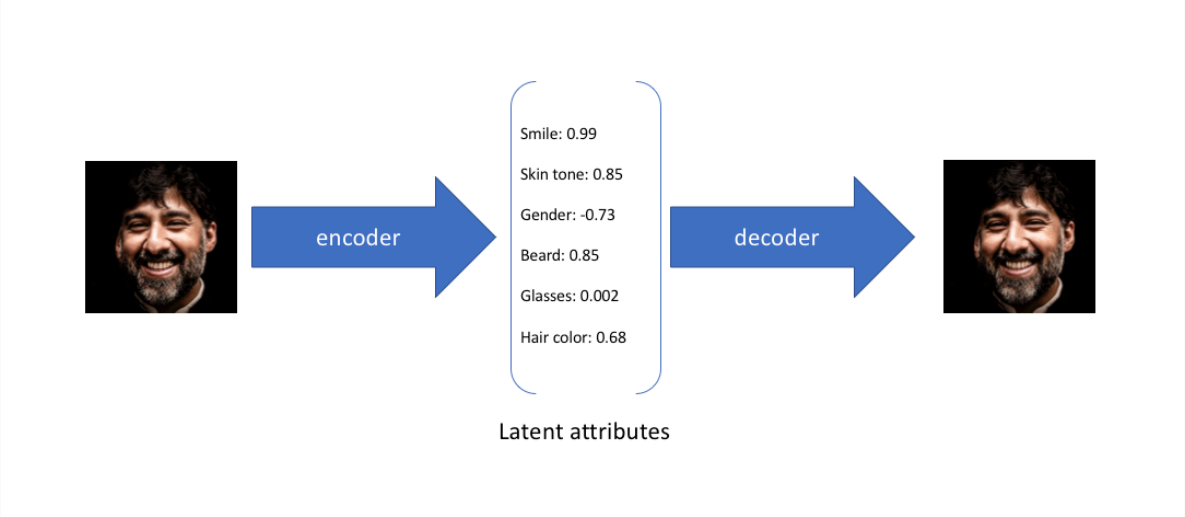
La variable c suele ser un vector *One-Hot* o un *Embedding*.

- En el Encoder, se concatena c con la entrada x.
- En el Decoder, se concatena c con el vector latente z.

De esta manera, la red aprende a separar la "identidad" del objeto (clase c) de su "estilo" o detalles menores (ruido z).

Aplicaciones

Una aplicación de los VAEs es la de poder generar muestras teniendo cierto control sobre lo que generamos.

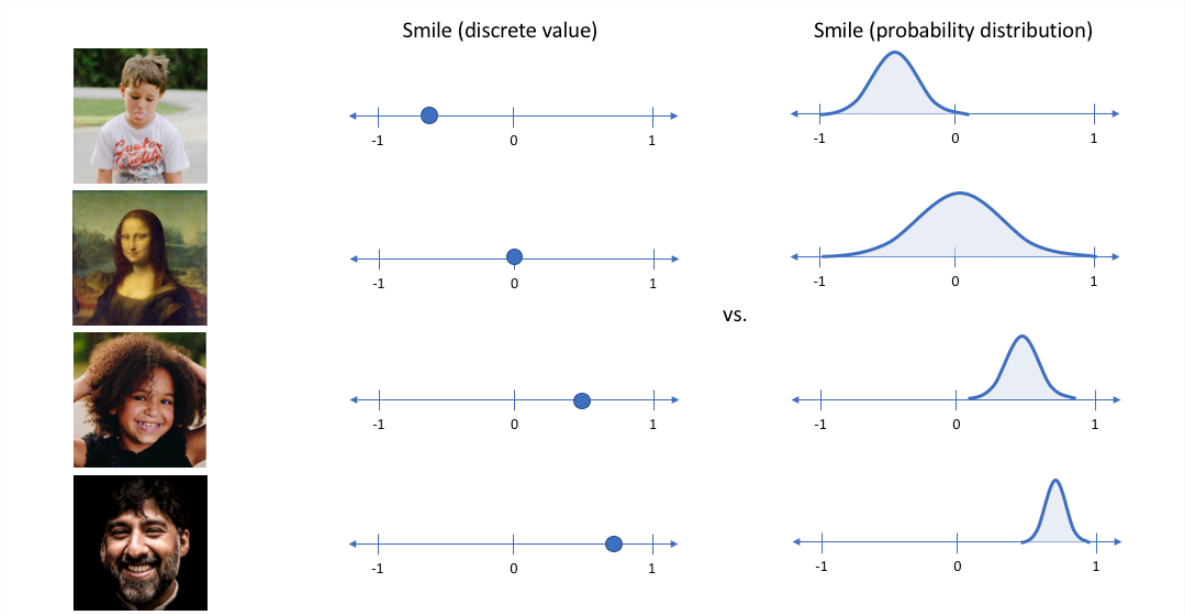


Ejemplo de generación de caras controlando los atributos (fuente).

En los VAEs el espacio latente está entrelazado. Conditional VAE.

Aplicaciones

¿Cómo se comportarían un AE y un VAE a la hora de caracterizar la sonrisa de una imagen?

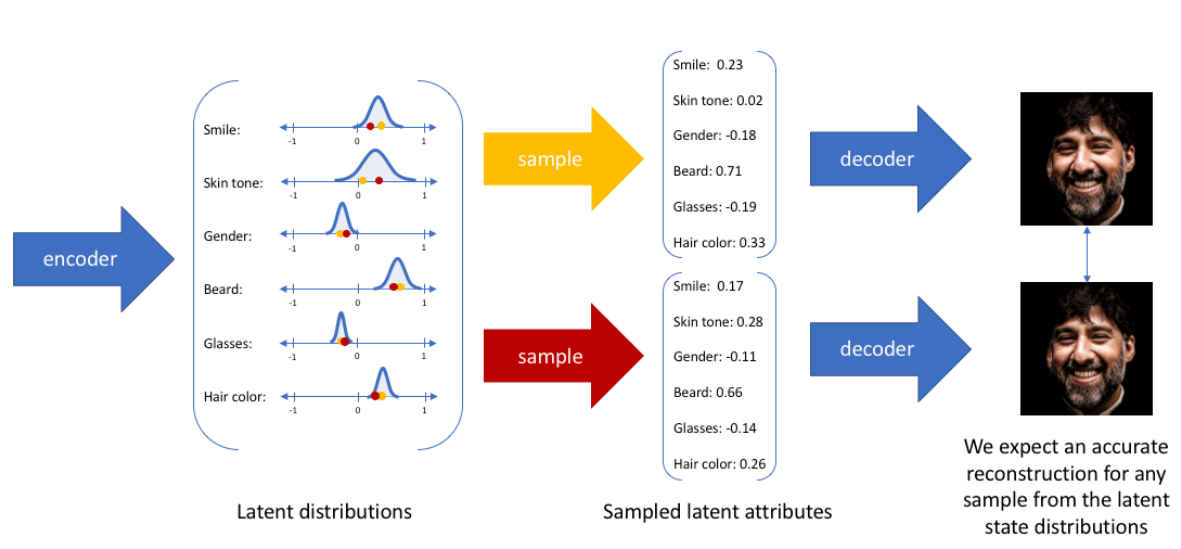


Ejemplo de espacio latente en AEs vs VAEs (fuente).

Aplicaciones

Al contrario que con los AEs, con los VAEs podemos obtener muestras nuevas diferentes a las de entrenamiento.

Por ejemplo, si muestreamos dos veces con valores similares, deberemos obtener muestras similares:



Ejemplo de generación de dos caras similares controlando los atributos (fuente).

El debate: ¿Supervisado o No Supervisado?

Tradicionalmente, el *Machine Learning* se divide en dos grandes ramas:

- **Supervisado:** Tenemos datos X y etiquetas Y anotadas por humanos. Buscamos $p(y|x)$[2].
- **No Supervisado:** Solo tenemos datos X y buscamos patrones ocultos (e.g., *clustering*, reducción dimensional). Buscamos $p(x)$[3].

¿Dónde encajan los Autoencoders?

- A menudo se clasifican como no supervisados porque no hay Y.
- Mecánicamente son supervisados: usan una función de pérdida contra un objetivo exacto (ground truth) y usan retropropagación.

[2] O visto de otro modo $f(X) \to Y$

[3] O también se puede ver como $f(X)$

La solución: Aprendizaje Autosupervisado

Para resolver esta paradoja terminológica, Yann LeCun [6] y otros pioneros de la IA popularizaron el término Aprendizaje Autosupervisado (Self-Supervised Learning)[4].

En este paradigma, los propios datos proporcionan la supervisión. El modelo retiene o corrompe una parte de la información y usa el resto para predecir lo que falta.

En el caso de los Autoencoders:

- **AE Clásico:** El objetivo (etiqueta) es el propio dato de entrada. $y = x$.
- **Denoising AE:** El dato de entrada es corrupto ($\tilde{x}$) y la "etiqueta" es el dato original limpio (x).

[4] Que, además, es un término muy rimbombante, lo cual siempre mola en IA.

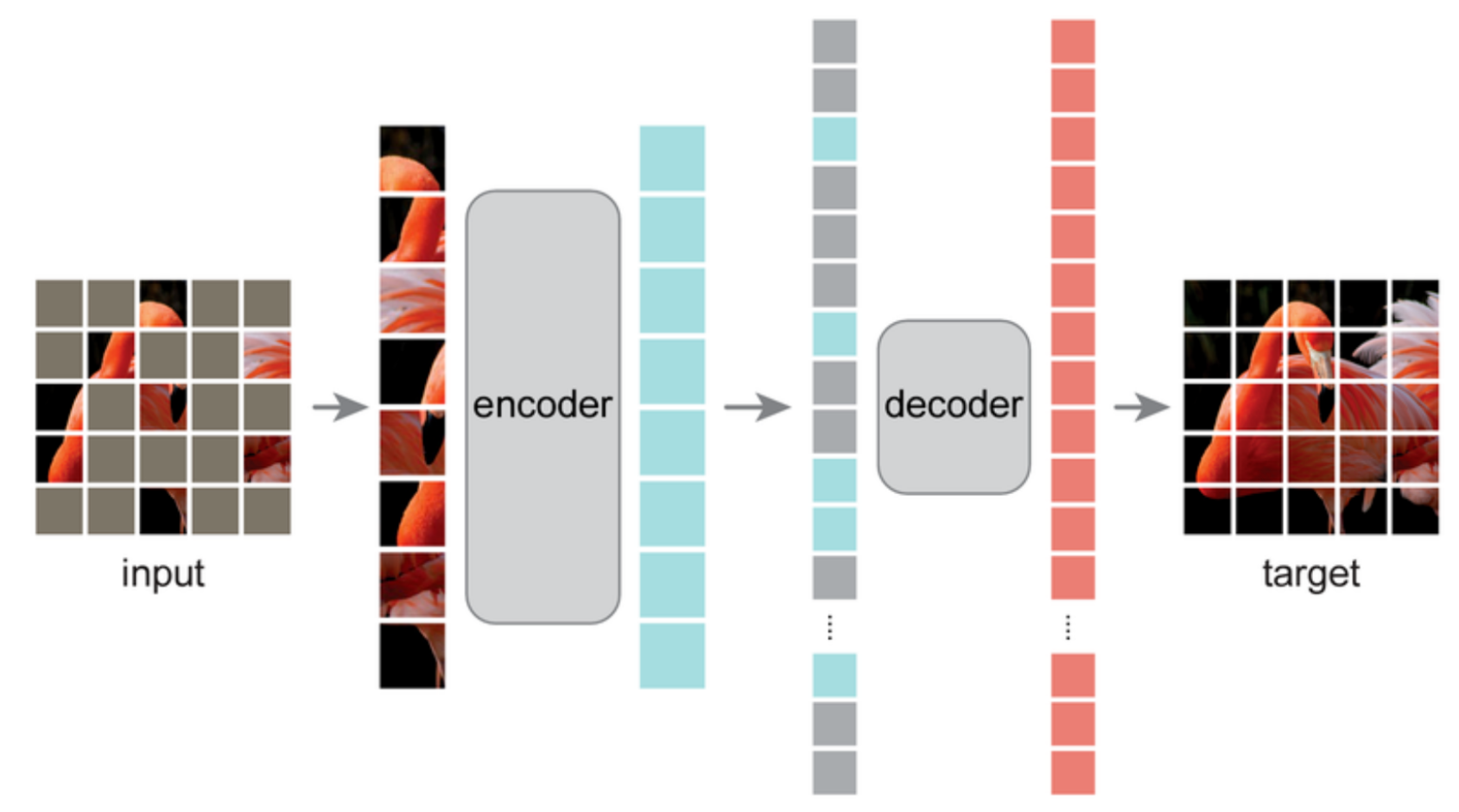
Impacto del Aprendizaje Autosupervisado

El Aprendizaje Autosupervisado es el paradigma dominante en la Inteligencia Artificial moderna, ya que permite entrenar modelos masivos aprovechando la inmensa cantidad de datos sin etiquetar que hay en internet.

Ejemplos clave que nacen de esta misma idea:

- **Procesamiento de Lenguaje (e.g., BERT):** Se enmascara el 15% de las palabras de una frase (como un ruido) y la red debe predecir cuáles eran. Es un Masked Autoencoder de texto.
- **Modelos Autorregresivos (e.g., GPT):** La red lee las primeras N palabras y su "etiqueta" (supervisión) es simplemente la palabra $N + 1$ que ya venía en el texto original.
- **Visión por Computador:** Ocultar parches enteros de una imagen y pedirle a un *Vision Transformer* que los reconstruya.

Aprendizaje Autosupervisado



[7]

I now call it “self-supervised learning”, because “unsupervised” is both a loaded and confusing term. [...] Self-supervised learning uses way more supervisory signals than supervised learning, and enormously more than reinforcement learning. That’s why calling it “unsupervised” is totally misleading.

Yann LeCun - Recent Advances in Deep Learning (2019) [8]

Generación de imágenes con Variational Autoencoders

Referencias i

[1] Julien Despois (HACKERNOON).
Mnist latent space image.
https://hackernoon.com/latent-space-visualization-deep-learning-bits-2-bd09a46920df, February 23rd, 2017.
[Online; accessed July, 2026].

[2] Diederik P Kingma and Max Welling.
Auto-encoding variational bayes.
arXiv preprint arXiv:1312.6114, 2013.

[3] Carl Doersch.
Tutorial on variational autoencoders.
arXiv preprint arXiv:1606.05908, 2016.

Referencias ii

[4] Kihyuk Sohn, Honglak Lee, and Xinchen Yan.
Learning structured output representation using deep conditional generative models.
Advances in neural information processing systems, 28, 2015.

[5] Khouloud Abdelli, Helmut Grießer, Christian Neumeyr, Robert Hohenleitner, and Stephan Pachnicke.
Degradation prediction of semiconductor lasers using conditional variational autoencoder.
Journal of Lightwave Technology, 40(18):6213–6221, 2022.

[6] Yann LeCun and Ishan Misra.
Self-supervised learning: The dark matter of intelligence (meta ai blog).
https://ai.facebook.com/blog/self-supervised-learning-the-dark-matter-of-intelligence/, 2021.

Referencias iii

[7] Yuchong Yao, Nandakishor Desai, and Marimuthu Palaniswami.
Masked contrastive representation learning.
arXiv preprint arXiv:2211.06012, 2022.

[8] Yann LeCun.
Yann lecun facebook publication (facebook).
https://www.facebook.com/722677142/posts/10155934004262143/, 2019.

Citación

Cita APA: Iglesias, G. and Talavera, E. (2026, noviembre 7). *Variational Autoencoders* [Diapositivas de la asignatura Métodos Generativos]. GitHub. *https://guillermoih.github.io/M-todos-Generativos/Variational_Autoencoders.pdf*

Tema de las diapositivas "Metropolis" lincenciado bajo LPPL 1.3c

BibTeX

```
@slides{Variational_Autoencoders_MG_2026,
   author = {Iglesias, Guillermo and Talavera, Edgar},
   title = {Variational Autoencoders [Diapositivas de la asignatura Métodos Generativos]},
   year = {2026},
   month = {11},
   howpublished={\url{
          https://guillermoih.github.io/M-todos-Generativos/Variational_Autoencoders.pdf}},
   note = {[Online; accessed 7 de noviembre de 2026]},
   organization={ETSISI. Universidad Politécnica de Madrid}
}
```